UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

WORKSHEET 5

Student Name: Aradhya Sharma **UID: 25MCC20042**

Branch: MCA (CCD) **Section/Group: 25MCC-101A**

Semester: 2 **Date of Performance: 24 FEB 2026**

Subject Name: Technical training **Subject Code: 25CAP-652**

1. AIM

To gain hands-on experience in creating and using **cursors** for **row-by-row processing** in PostgreSQL, enabling **sequential access** and **controlled manipulation** of query results for implementing complex business logic.

---

2. OBJECTIVES

- To understand sequential data access using cursors
- To perform row-level manipulation using procedural logic
- To learn proper cursor lifecycle management
- To handle cursor termination and empty result sets
- To apply cursor-based logic to real-world business scenarios

---

3. SOFTWARE REQUIREMENTS

Operating System:
- Windows 10 / 11 / Linux / macOS

Database Management System:
- PostgreSQL (version 12 or higher)

SQL Interface:
- pgAdmin 4

OR
- PostgreSQL Command Line (psql)

4. PROCEDURE FOR EXPERIMENT

Step 1: Implementing a Simple Forward-Only Cursor

Objective:
To fetch employee records one by one using a cursor and display them sequentially.

```
CREATE TABLE employee (
   emp_id INT PRIMARY KEY,
   emp_name TEXT,
   salary NUMERIC,
   experience INT,
   performance_rating INT
);

INSERT INTO employee VALUES
(101, 'Amit',   42000, 6, 4),
(102, 'Priya',  36000, 3, 3),
(103, 'Rahul',  55000, 8, 5),
(104, 'Neha',   30000, 2, 2),
(105, 'Arjun',  48000, 5, 4),
(106, 'Rohit',  39000, 4, 3),
(107, 'Kiran',  46000, 7, 4),
(108, 'Sneha',  34000, 3, 3),
(109, 'Vikas',  51000, 9, 5),
(110, 'Pooja',  32000, 1, 2);

DO $$
DECLARE
   emp_cursor CURSOR FOR
      SELECT emp_id, emp_name, salary FROM employee;

   v_id INT;
   v_name TEXT;
   v_salary NUMERIC;
BEGIN
```

```
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_id, v_name, v_salary;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'ID: %, Name: %, Salary: %',
                     v_id, v_name, v_salary;
    END LOOP;

    CLOSE emp_cursor;
END $$;
```

Output:

```
NOTICE:  ID: 101, Name: Amit, Salary: 48300.00
NOTICE:  ID: 102, Name: Priya, Salary: 37800.00
NOTICE:  ID: 103, Name: Rahul, Salary: 63250.00
NOTICE:  ID: 104, Name: Neha, Salary: 31500.00
NOTICE:  ID: 105, Name: Arjun, Salary: 55200.00
NOTICE:  ID: 106, Name: Rohit, Salary: 40950.00
NOTICE:  ID: 107, Name: Kiran, Salary: 52900.00
NOTICE:  ID: 108, Name: Sneha, Salary: 35700.00
NOTICE:  ID: 109, Name: Vikas, Salary: 58650.00
NOTICE:  ID: 110, Name: Pooja, Salary: 33600.00
DO

Query returned successfully in 95 msec.
```

Step 2: Complex Row-by-Row Manipulation

Objective:
To update employee salaries using cursor-based conditional logic based on experience and performance.

Business Logic:
- Experience ≥ 5 and Performance ≥ 4 → 15% increment
- Otherwise → 5% increment

```
DO $$
DECLARE
   emp_cursor CURSOR FOR
      SELECT emp_id, salary, experience, performance_rating
      FROM employee
      FOR UPDATE;

   v_id INT;
   v_salary NUMERIC;
   v_exp INT;
   v_perf INT;
BEGIN
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO v_id, v_salary, v_exp, v_perf;
      EXIT WHEN NOT FOUND;

      IF v_exp >= 5 AND v_perf >= 4 THEN
         UPDATE employee
         SET salary = salary * 1.15
         WHERE CURRENT OF emp_cursor;
      ELSE
         UPDATE employee
         SET salary = salary * 1.05
         WHERE CURRENT OF emp_cursor;
      END IF;
   END LOOP;

   CLOSE emp_cursor;
END $$;
```

Output:

```
DO

Query returned successfully in 94 msec.
```

Step 3: Exception and Status Handling

Objective:
To handle empty result sets and ensure safe cursor termination.

```
DO $$

DECLARE

   emp_cursor CURSOR FOR

      SELECT emp_id FROM employee;

   v_id INT;

   row_count INT := 0;

BEGIN

   OPEN emp_cursor;

   LOOP

      FETCH emp_cursor INTO v_id;

      EXIT WHEN NOT FOUND;

      row_count := row_count + 1;

      RAISE NOTICE 'Employee ID: %', v_id;

   END LOOP;

   IF row_count = 0 THEN

      RAISE EXCEPTION 'No employee records found in the table.';

   END IF;

   CLOSE emp_cursor;
```

```
EXCEPTION
    WHEN OTHERS THEN
        CLOSE emp_cursor;
        RAISE NOTICE 'Exception handled safely: %', SQLERRM;
END $$;
```

Output:

```
NOTICE:  Employee ID: 101
NOTICE:  Employee ID: 102
NOTICE:  Employee ID: 103
NOTICE:  Employee ID: 104
NOTICE:  Employee ID: 105
NOTICE:  Employee ID: 106
NOTICE:  Employee ID: 107
NOTICE:  Employee ID: 108
NOTICE:  Employee ID: 109
NOTICE:  Employee ID: 110
DO

Query returned successfully in 122 msec.
```

Result Verification

	emp_id [PK] integer	emp_name text	salary numeric	experience integer	performance_rating integer
1	101	Amit	55545.0000	6	4
2	102	Priya	39690.0000	3	3
3	103	Rahul	72737.5000	8	5
4	104	Neha	33075.0000	2	2
5	105	Arjun	63480.0000	5	4
6	106	Rohit	42997.5000	4	3
7	107	Kiran	60835.0000	7	4
8	108	Sneha	37485.0000	3	3
9	109	Vikas	67447.5000	9	5
10	110	Pooja	35280.0000	1	2

Learning Outcome

This experiment demonstrates effective use of **cursors in PostgreSQL** for sequential data access and row-by-row processing.
Students gain strong understanding of cursor lifecycle management and exception handling, which is essential for:

- Payroll systems
- Data migration tasks
- Backend application development
- Technical interviews and placements